

An Analysis of the Ajtai, Kumar, Sivakumar (AKS) Sieving Algorithm

Fernando Flores Hernandez

Instructor: Prof. Stanisław Radziszowski

Spring 2020

Abstract

Significant advancements have been made in the field of cryptography and math that have shown that certain problems once thought to be very hard to solve can in fact be solved efficiently with quantum computers. Shor's algorithm is an example of this, efficiently solving the Factoring Problem and the Discrete Logarithm Problem [9]. Many modern cryptosystems such as RSA and ElGamal rely on the infeasibility of these problems to provide the security that they do.

As of the time of this paper, it is still an open question whether anyone has built a quantum computer with quantum supremacy. Google, with help from NASA, has claimed to have done it [2], but it is disputed. Lattice-based cryptography is being studied to create the next generation of cryptosystems. Many of these cryptosystems rely on the hardness of lattice problems such as the Closest Vector Problem (CVP) and the Shortest Vector Problem (SVP) to guarantee their security.

In this paper we will explore the Shortest Vector Problem, the Closest Vector Problem, Babai's Approximation Algorithm, the Lenstra–Lenstra–Lovász algorithm, and how they all relate to AKS, the first (randomized) algorithm to solve exact SVP in time $2^{O(n)}$. We describe the algorithm, sketch why it works, and explain why it is not practical.

1 Preliminaries

We begin by defining some of the mathematical notation used throughout the paper.

1.1 Lattice

Let $B = \{b_1, \dots, b_n\}$ be a basis of n linearly independent vectors in $\mathbb{R}^n$. The lattice generated by B , denoted $\mathcal{L} = \mathcal{L}(B)$, is the set of all linear combinations of the basis vectors with *integer* coefficients:

$$\mathcal{L}(B) = \left\{ \sum_{i=1}^n a_i b_i \mid a_i \in \mathbb{Z} \right\}.$$

A lattice has many different bases, and some are much nicer than others. A basis of long, nearly parallel vectors describes exactly the same lattice as a basis of short, nearly orthogonal ones, and this difference is the whole reason the problems below are hard (Figure 1).

1.2 Fundamental Parallelepiped

The fundamental parallelepiped (or region) of a basis is the set of points we get by using coefficients in a window of length one instead of integers. I use the centered version:

$$\mathcal{P}(B) = \left\{ \sum_{i=1}^n x_i b_i \mid -\frac{1}{2} \leq x_i < \frac{1}{2} \right\}.$$

Every point of $\mathbb{R}^n$ can be written uniquely as a point of $\mathcal{P}(B)$ plus a lattice vector. Any point in $\mathcal{P}(B)$ has length at most $\frac{1}{2} \sum_i \|b_i\| \leq \frac{n}{2} \max_i \|b_i\|$.

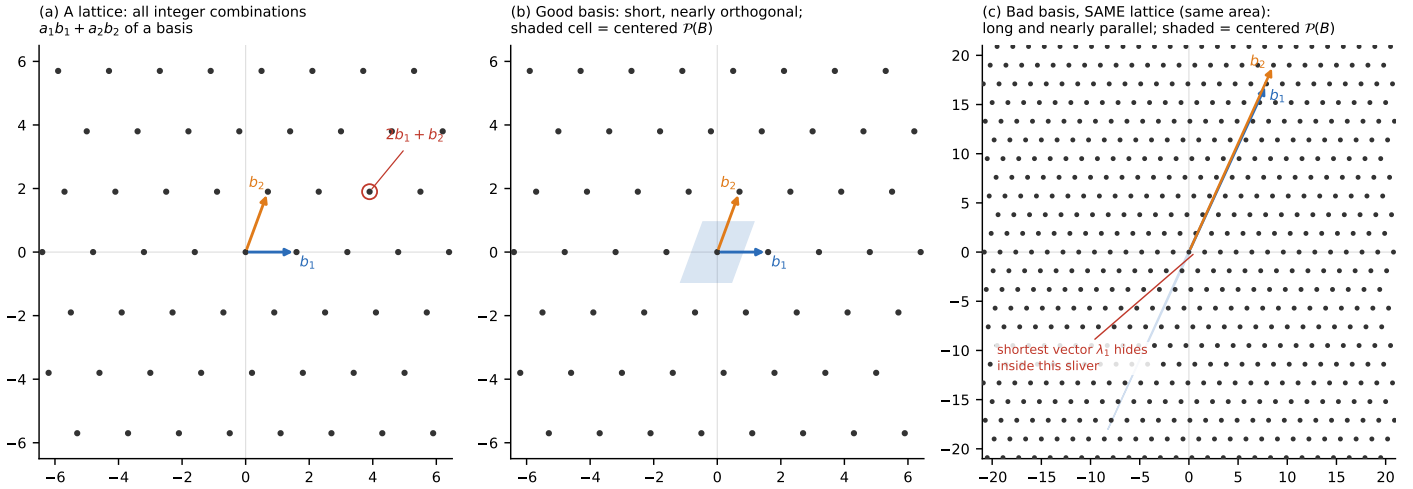


Figure 1: A two-dimensional lattice. (a) The lattice is every integer combination of a basis. (b) With a good basis (short, nearly orthogonal vectors) the shaded (centered) fundamental parallelepiped $\mathcal{P}(B)$ is small and the short lattice vectors are easy to see. (c) The same lattice described by a bad basis (long, nearly parallel vectors): $\mathcal{P}(B)$ is a long sliver and the shortest vector λ_1 is hidden inside it.

1.3 Closest Vector Problem (CVP)

The Closest Vector Problem asks, given a lattice with basis B and any arbitrary target vector $\vec{w} \in \mathbb{R}^n$, to find a lattice vector $\vec{u} \in \mathcal{L}(B)$ that minimizes $\|\vec{u} - \vec{w}\|$. To rephrase: we are looking for the lattice point that is closest to the given point.

1.4 Shortest Vector Problem (SVP)

The Shortest Vector Problem asks, given a basis B , to find a *nonzero* vector in $\mathcal{L}(B)$ of minimum length. Write $\lambda_1(\mathcal{L}) = \min_{\vec{v} \in \mathcal{L} \setminus \{0\}} \|\vec{v}\|$ for that minimum length. The word nonzero matters, since otherwise the zero vector wins trivially.

SVP is not harder than CVP in the usual sense (there is a reduction from SVP to CVP), so CVP is the more general problem. Whether a lattice is “easy” depends on the basis we are handed. For the integer lattice $\mathbb{Z}^n$ with the standard basis the shortest vector is obvious, but the same lattice handed to us with a skewed basis is not obvious at all. In two dimensions Lagrange–Gauss reduction solves SVP quickly, and it gets much harder as n increases.

2 The AKS Algorithm

2.1 Overview

From my understanding the algorithm is composed of four steps, followed by reading off the answer. First there is (optional) preprocessing of our input, which is the lattice we are working in. Second, some parameters are set up that determine how many points will be sampled and how large the region we start sieving in is. Third, we sample points from a small ball around the origin and turn each of them into a point in the fundamental parallelepiped together with a lattice vector that remembers how we got there. Fourth, we put those points through the sieve over and over, shrinking the radius they live in each time, until they all sit in a small ball. At that point the differences of the remembered lattice vectors contain the shortest vector of the lattice.

Throughout, $\xi = 2$ denotes the radius of the ball we sample from and R denotes the radius of the ball that the current points live in. The analysis assumes the lattice has been scaled so that $\xi/2 \leq \lambda_1 \leq c\xi$ for some constant $c < 2$ (say $c = 3/2$). The upper bound matters because it makes two balls of radius ξ , centered a shortest vector apart, overlap in a decent chunk of their volume ($2^{-O(n)}$ of it). Just having $\lambda_1 < 2\xi$ is not enough, since the overlap shrinks quickly as λ_1 gets close to 2ξ . The lower bound keeps the number of short lattice vectors under control (Section 3.4). We get this scaling by guessing λ_1 pretty closely, to within about $1 + 1/n$, and rescaling, at the cost of polynomially many runs. I don't go through the details of this here. I use the constants from the version of the algorithm I implemented and do not try to tighten them.

2.2 Preprocessing

Preprocessing of the input is completely optional. In preprocessing we run the LLL algorithm [4] on the basis. LLL does not change the lattice; it uses the Gram–Schmidt process as a tool to output a *reduced* basis of the same lattice, consisting of short, nearly orthogonal vectors (with the shortest basis vector within a factor of $2^{(n-1)/2}$ of λ_1). A shorter basis gives a smaller starting radius R_0 below. Since R_0 only enters the sample size and the number of rounds through $\log R_0$, the gain in the worst-case bound is modest, but it also makes the Babai step in the sampling routine produce shorter points. I never got to measure how much this matters in practice.

2.3 Initialization

In this portion of the algorithm, variables are set to figure out the number of points to sample and the initial radius R_0 , which is an upper bound on the length of any point in the region we start sieving in:

$$R_0 = n \cdot \max_i \|b_i\| \quad \text{sieving initial radius} \quad (1)$$

$$N = 2^{8n} \log R_0 \quad \text{number of points to sample} \quad (2)$$

Since points of $\mathcal{P}(B)$ have length at most $\frac{n}{2} \max_i \|b_i\|$, this R_0 is a valid bound, though a factor of two larger than necessary.

2.4 Point Sampling Routine

In the point sampling routine we sample N points x_i from the ball $B(0, \xi)$ of radius $\xi = 2$ around the origin in $\mathbb{R}^n$. For each one we compute a lattice vector z_i and a perturbed point y_i . The

steps are essentially Babai's rounding procedure [3]. For each sampled point x_i we solve for its coordinates in the basis:

$$Bv = x_i \quad \text{solve for } v \text{ and round every entry of } v \text{ to the nearest integer} \quad (3)$$

The rounded coordinates give a lattice vector, which is guaranteed to be in the lattice since its coordinates are integers:

$$z_i = B\lfloor v \rfloor \in \mathcal{L}(B) \quad (4)$$

The perturbed point is what is left over after subtracting the lattice vector:

$$y_i = x_i - z_i = B(v - \lfloor v \rfloor) \in \mathcal{P}(B). \quad (5)$$

So y_i lies in the fundamental parallelepiped, at distance at most $R_0/2$ from the origin, and, most importantly for what follows,

$$x_i - y_i = z_i \in \mathcal{L}(B) \quad (\text{the invariant we keep for the rest of the algorithm}). \quad (6)$$

We keep the triples (x_i, y_i, z_i) , for $i = 1, \dots, N$. Figure 2 shows the step in a small example, and Figure 3 shows what it does to a lattice given by a bad basis: the sampled points all sit in a tiny ball, but the perturbed points can be as far from the origin as $R_0/2$.

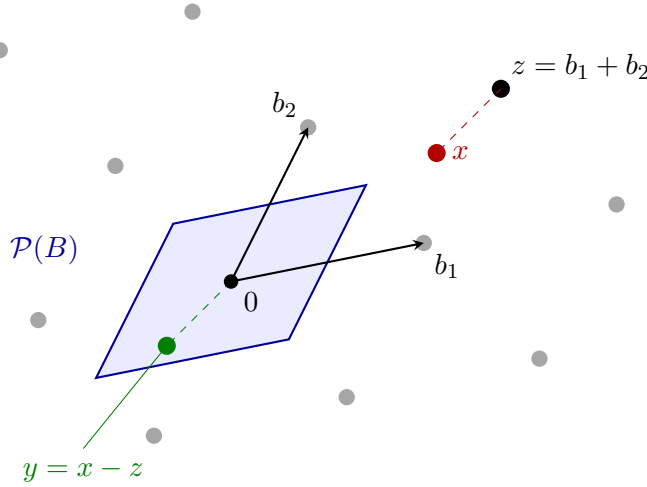


Figure 2: The Babai step in two dimensions. The sampled point x has coordinates $v \approx (0.82, 0.63)$ in the basis, which round to $(1, 1)$, so $z = b_1 + b_2$. Subtracting z moves x into the fundamental parallelepiped as $y = x - z$.

2.5 Sieving

Now this is where most of the action happens. Each round takes the current radius R and the current points y_i (all of length at most R) and does the following.

1. **Choose centers.** Go through the points in order. If y_i is within distance $R/2$ of a center that was already chosen, we map i to that center $c(i)$. Otherwise y_i becomes a new center.
2. **Remove the centers.** The centers are thrown away, along with their x and z .

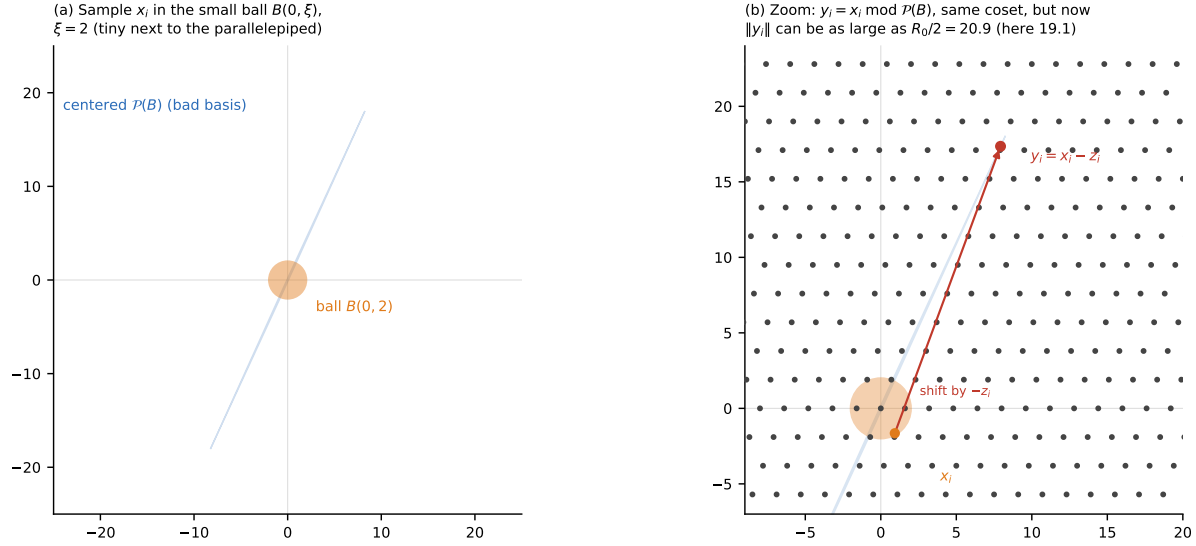


Figure 3: Sampling on a lattice with a bad basis. (a) The sampled points x_i live in the tiny ball $B(0, \xi)$, which is nothing next to the parallelepiped $\mathcal{P}(B)$. (b) Zoomed in: subtracting the lattice vector z_i leaves the same coset, $y_i = x_i - z_i$, but now $\|y_i\|$ can be as large as $R_0/2$.

3. Shift everyone else by their center's z . For every remaining i with center $j = c(i)$ we replace

$$y_i \leftarrow y_i + z_j, \quad z_i \leftarrow z_i - z_j. \quad (7)$$

The update is exactly $y_i \leftarrow y_i - y_j + x_j$, because $y_j = x_j - z_j$. It keeps the invariant $x_i - y_i = z_i \in \mathcal{L}(B)$ (the new z_i is still a difference of lattice vectors), and the new y_i is the old distance to the center, $\|y_i - y_j\| \leq R/2$, plus a sampled point of length at most ξ . So after one round all points live in a ball of radius

$$R \leftarrow \frac{R}{2} + \xi. \quad (8)$$

Because the centers are pairwise more than $R/2$ apart and all live in a ball of radius R , there are at most 5^n of them (see the analysis below), so each round removes at most 5^n points. Figure 4 sketches one round and Figure 5 shows one on the toy lattice.

2.6 Answer

We repeat the sieving round until $R \leq 6$. After that we are still left with at least 2^{7n} points (see below), and each remaining point satisfies $\|y_i\| \leq 6$ and $\|x_i\| \leq 2$. Hence the lattice vectors satisfy $\|z_i\| = \|x_i - y_i\| \leq 6 + 2 = 8$. That is where the bound of 8 comes from: it is a bound on the lattice vectors z_i , not on the y_i . We then compute the differences $z_i - z_j$ of pairs of remaining lattice vectors, keep the shortest *nonzero* one, return it, and we're done. The next section explains why that vector is a shortest vector of the lattice (Figure 7).

3 Analysis

This section sketches why the algorithm behaves the way the previous section claims. The full proof is in the original paper [1] and, in a cleaner form, in Regev's lecture notes [8]; the constants

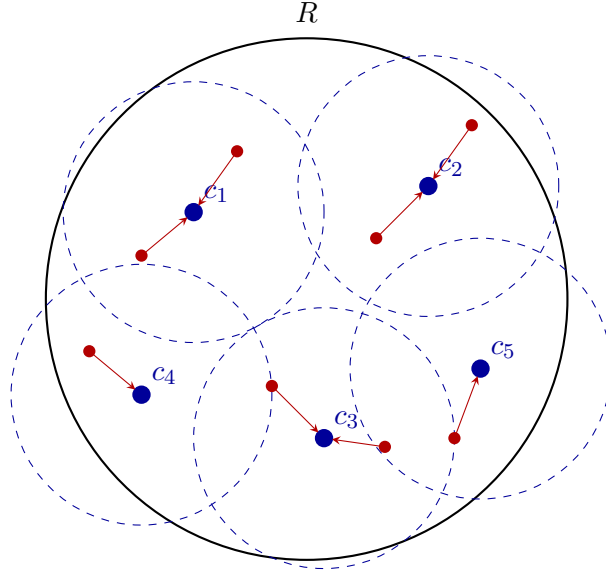


Figure 4: One round of the sieve, drawn in two dimensions. The centers c_j (blue) are pairwise more than $R/2$ apart; every other point (red) lies within $R/2$ of some center and is moved by the update above to sit within $R/2 + \xi$ of the origin. The centers are then discarded. The arrows only show which center each point goes with. The real move is the shift by $z_c = x_c - y_c$.

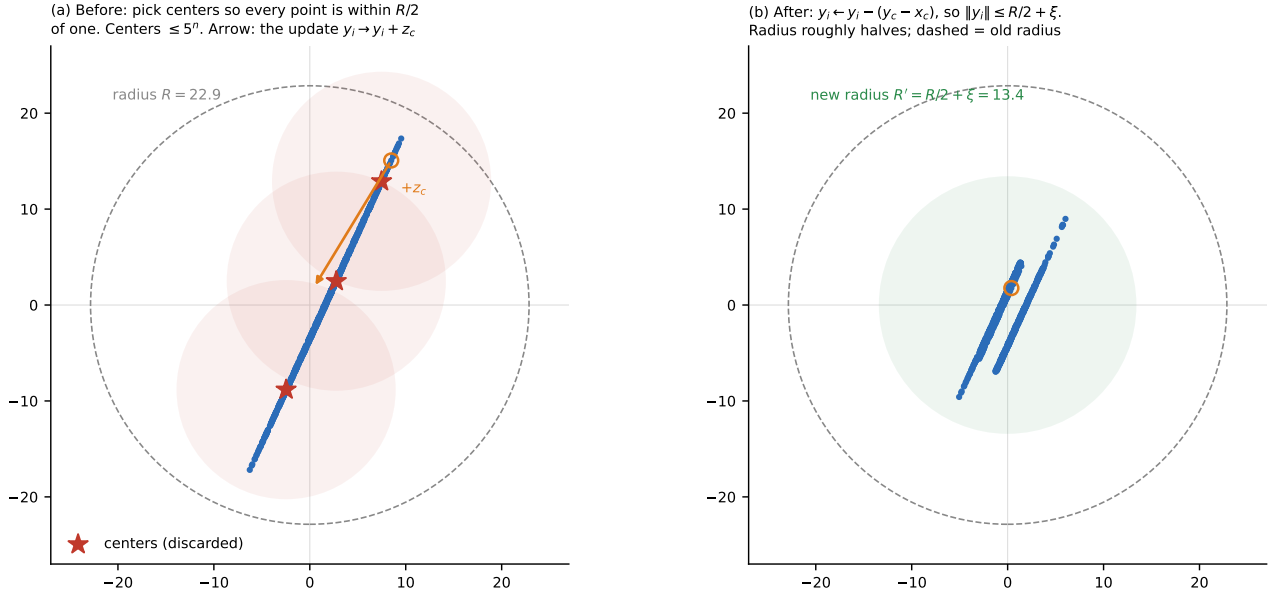


Figure 5: One sieve round on the toy lattice. (a) Before: the centers (stars) are chosen so that every point is within $R/2$ of one; the arrow shows the update $y_i \rightarrow y_i + z_c$ of one point (with $z_c = x_c - y_c$). (b) After: $y_i \leftarrow y_i - (y_c - x_c)$, and the radius has dropped from $R = 22.9$ to $R' = R/2 + \xi = 13.4$ (dashed = old radius).

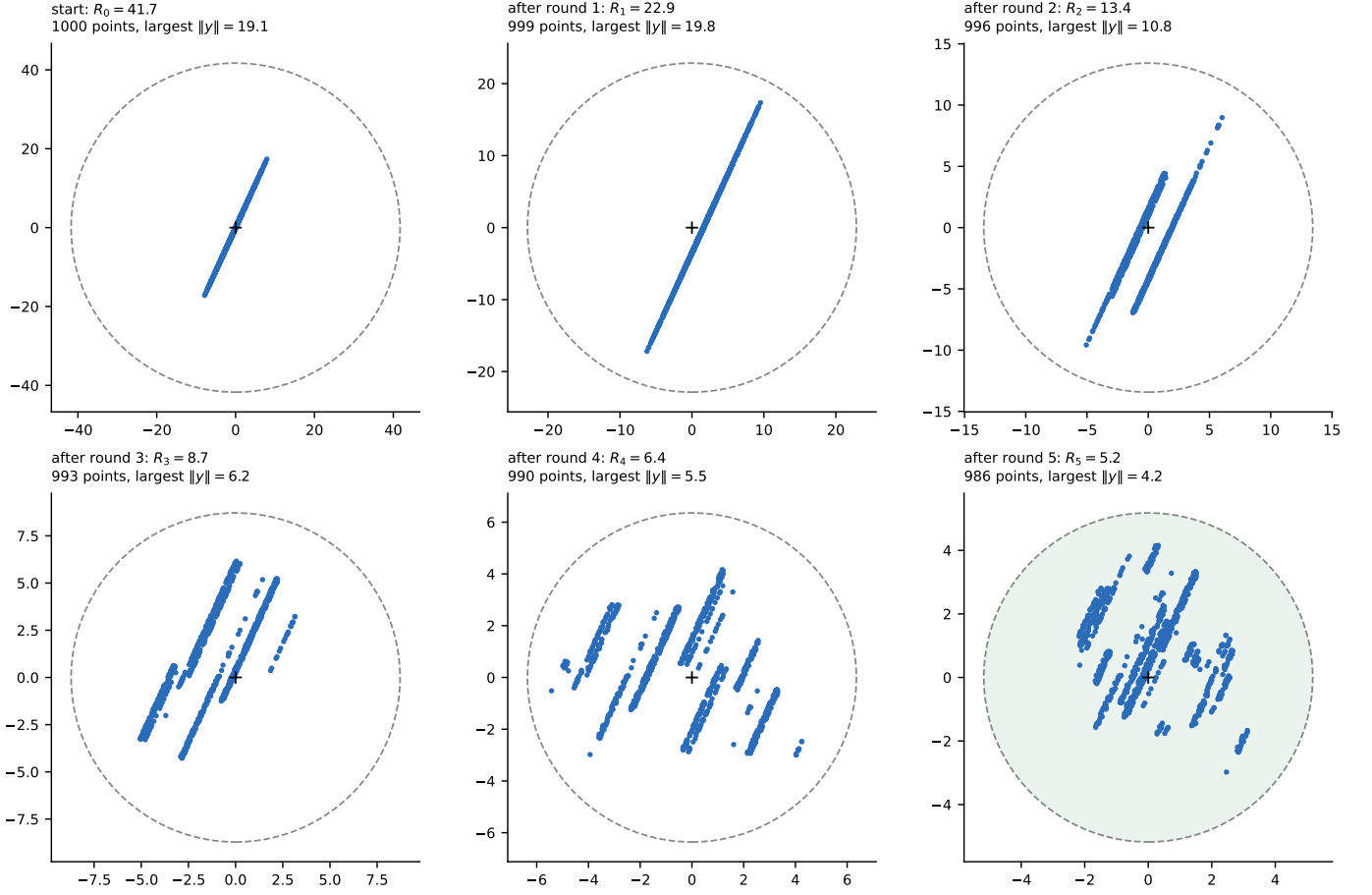


Figure 6: Repeating the sieve on the toy lattice, 1000 points. The radius goes from $R_0 = 41.7$ down to $R_5 = 5.2$, and the thin sliver that $\mathcal{P}(B)$ starts as is folded up into a compact cloud around the origin.

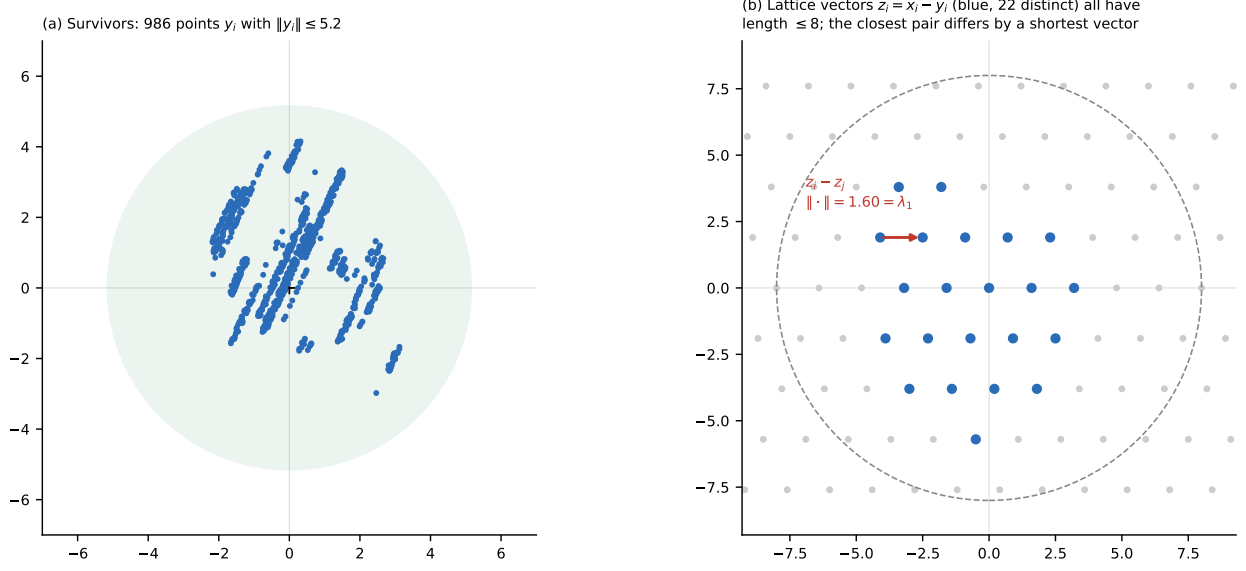


Figure 7: Reading off the answer on the toy lattice. (a) The 986 surviving points y_i all lie within radius 5.2 of the origin. (b) The lattice vectors $z_i = x_i - y_i$ (blue, 22 distinct) all have length ≤ 8 ; the difference of the closest pair is a shortest vector, of length $\lambda_1 = 1.60$.

I use differ slightly from theirs.

3.1 The invariant

Throughout the algorithm, every triple satisfies $z_i = x_i - y_i \in \mathcal{L}(B)$. It holds after sampling by construction, and each sieve update preserves it because it only ever adds or subtracts lattice vectors from z_i . Nothing else is needed for the output to always be an element of the lattice. The interesting question is only why it is a *shortest* one.

3.2 The radius shrinks to about 2ξ

Each round maps $R \mapsto R/2 + \xi$. Writing R_k for the radius after k rounds, we get $R_k - 2\xi = (R_0 - 2\xi)/2^k$, so R_k converges to the fixed point $2\xi = 4$ and drops below 6 after $k = O(\log R_0)$ rounds. That is why the sample size only needs a $\log R_0$ factor. In the toy run the radius goes 41.7, 22.9, 13.4, 8.7, 6.4, 5.2 and the loop stops after five rounds (Figure 8a).

3.3 Each round removes at most 5^n points

The centers chosen in a round are pairwise more than $R/2$ apart, so the balls of radius $R/4$ around them are disjoint. All of these balls fit inside a ball of radius $R + R/4 = 5R/4$. Comparing volumes, the number of centers is at most $(5R/4)^n / (R/4)^n = 5^n$. Over all rounds we lose at most $5^n \log R_0$ points, which is tiny compared to $N = 2^{8n} \log R_0$. In particular, at least $2^{8n} \log R_0 - 5^n \log R_0 \geq 2^{7n}$ points survive until the end, matching the count in the previous section. In the toy run ($n = 2$, so at most $5^2 = 25$ centers per round) each round removes only between one and four of the 1000 points (Figure 8b). The toy run only uses $N = 1000$ points. For $n = 2$ the formula for N above would give about 3.5×10^5 (using $\log_2$), so the toy run is much smaller than what the real algorithm

asks for. It is just there to show how the sieve works, so the guarantee of $2^{7n} = 16384$ survivors does not apply to it.

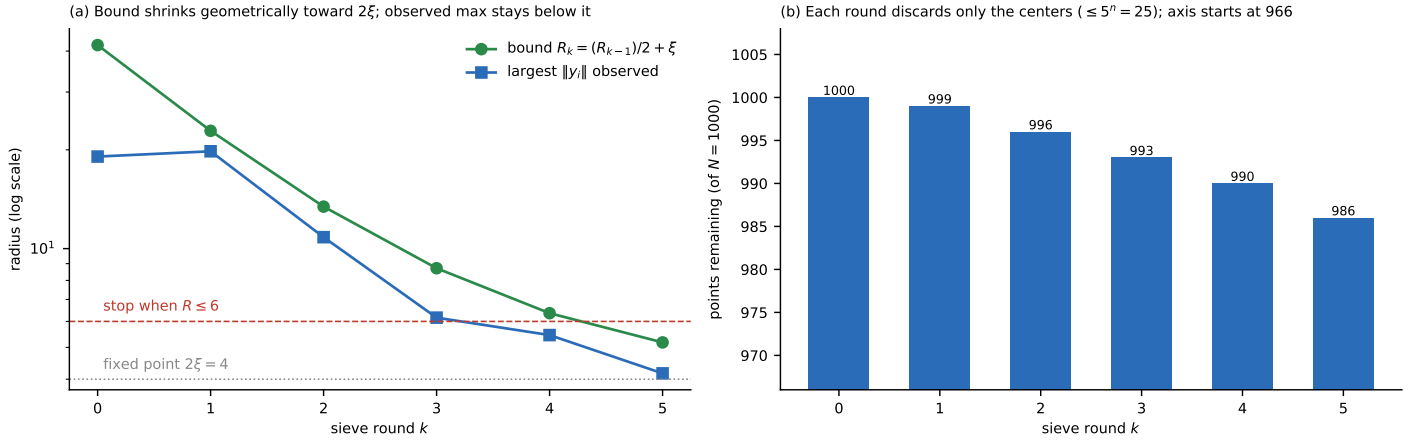


Figure 8: The two bounds, checked on the toy lattice. (a) The radius bound $R_k = R_{k-1}/2 + \xi$ and the largest $\|y_i\|$ actually observed; the bound shrinks geometrically toward the fixed point $2\xi = 4$, and the observed maximum stays below it. (b) Points remaining out of $N = 1000$: each round discards only the centers, far fewer than the $5^n = 25$ allowed.

3.4 Why the shortest vector shows up

Let $\vec{v}$ be a shortest nonzero lattice vector. The key observation is that the value of $y_i = x_i \bmod \mathcal{P}(B)$ only depends on the coset $x_i + \mathcal{L}$, and the sieve only ever looks at the y_i . Now take a sample x in the overlap $B(0, \xi) \cap B(\vec{v}, \xi)$. Then x and $x - \vec{v}$ are both possible samples (they are both in $B(0, \xi)$), they differ by a lattice vector, and so they produce exactly the same y . The algorithm cannot tell them apart, but the lattice vectors attached to them differ by exactly $\vec{v}$. Since the overlap is a $2^{-O(n)}$ fraction of the ball (this is where $\lambda_1 \leq c\xi$ with $c < 2$ is used), a large number of the 2^{7n} surviving points come from such pairs. There also aren't many places z can land. Each z_i is a lattice vector of length at most 8, and lattice points are at least $\lambda_1 \geq \xi/2 = 1$ apart. So balls of radius $\lambda_1/2$ around them don't overlap and all fit inside a ball of radius $8 + \lambda_1/2$. Comparing volumes like in Section 3.3, that gives at most $(1 + 16/\lambda_1)^n \leq 17^n$ possible values of z , which is far fewer than 2^{7n} . So a lot of the surviving points must share the same z . As far as I can tell, this is part of how the proof in [1] gets its collisions. The analysis in [1] shows that among the final lattice vectors there are two whose difference is $\pm\vec{v}$. No lattice vector shorter than $\vec{v}$ exists, so the shortest nonzero difference we output has length λ_1 . AKS is a randomized algorithm, so it only works with high probability over the random samples. This is a sketch, not a proof: the real argument has to control the distribution of the points through every round of the sieve.

3.5 Running time and why it is impractical

The space is $N = 2^{O(n)}$ points. In a round, every point is compared to at most 5^n centers, so sieving costs $O(N \cdot 5^n)$ distance computations per round, and the final step compares all pairs at cost $O(N^2)$. In total this is $2^{O(n)}$ time and space, which is why AKS was the first algorithm to solve exact SVP in single-exponential time. The constants, however, are enormous: with $N = 2^{8n} \log R_0$, already $n = 20$ needs about $2^{160} \approx 10^{48}$ points. This is why no one runs the algorithm as it

is written. Practical sieves such as the Nguyen–Vidick sieve [7] and the ListSieve of Micciancio–Voulgaris [5] keep the same basic idea (repeatedly combine points to obtain shorter ones), but with much better constants in the exponent and far fewer points. One thing worth knowing is that the $2^{0.415n}$ running time of Nguyen–Vidick depends on assumptions that are not proven. ListSieve of Micciancio–Voulgaris does have a proven bound of about $2^{3.2n}$. The other big algorithm in [5] is not a sieve at all. It is deterministic, uses the Voronoi cell of the lattice, and runs in time $2^{2n+o(n)}$. There is also proof of a $2^{n+o(n)}$ bound using discrete Gaussian sampling (Aggarwal, Dadush, Regev and Stephens-Davidowitz, STOC 2015).

4 Implementation

The implementation of the algorithm was not that hard, but debugging it was. I wasn’t able to fully debug it by the time this paper was due, but I got a pretty good idea of how the algorithm works in my attempts to debug the code. Rather than showing the unfinished code, the algorithm is given below as annotated pseudocode.

```

1 AKS(B):                                     # B = basis, columns b_1..b_n
2   n ← number of columns of B
3   R ← n * max_i ||b_i||                     # R_0
4   N ← 2^(8n) * log(R)
5   for i = 1..N:                             # sampling + Babai rounding
6     x_i ← uniform point in ball(0, 2)
7     v ← solve(B v = x_i)
8     z_i ← B * round(v)                     # lattice vector
9     y_i ← x_i - z_i                         # in P(B)
10  while R > 6:
11    centers ← []
12    for i in 1..(current number of points):
13      if exists j in centers with ||y_i - y_j|| ≤ R/2:
14        c(i) ← j
15      else:
16        centers.append(i)
17    # use the OLD y and z of the centers, then throw them away
18    for every i not in centers:
19      y_i' ← y_i + z_c(i)
20      z_i' ← z_i - z_c(i)
21    keep only the non-center points, with the updated y and z
22    R ← R/2 + 2
23  return shortest nonzero vector among { z_i - z_j : i != j }
```

The biggest takeaway from this was the sieving mechanism and how it filtered out points progressively. Two pitfalls are worth noting for anyone translating the pseudocode directly. Deleting entries from a list while iterating over its indices silently shifts everything after the deleted entry, and copying the list with a shallow copy leaves the nested vectors shared, so updating one point can change another. Building the new list of updated points from the old values in one pass avoids both.

I used numpy to perform most of the matrix multiplication and to find the Euclidean distance between points. I also installed a library called GAP that included the LLL Basis-Reduction algorithm that I was going to use to provide the reduced matrices to test the runtime on my algorithm.

5 Conclusion

The AKS sieve is a nice example of how a very simple idea (match each point with a nearby center, replace it with its difference from that center, and throw the centers away) can give the first single-exponential algorithm for an old problem. In order to improve this algorithm, we can certainly parallelize it to generate points, but the sieving routine is harder: which points become centers depends on the centers chosen before them, so the center selection is inherently sequential unless the algorithm is changed. More importantly, the sample size $2^{8n} \log R_0$ makes the algorithm impractical as stated, which is what motivated the practical sieves mentioned above. Next steps would be a in-depth analysis of how the algorithm would perform with LLL preprocessing and without.

In my free time I will look back at this and carry out a more complete version of the analysis as stated in the previous paragraph. The math behind the algorithm is pretty intense and I would like to be have a deeper understanding of it.

Acknowledgements

Some of the graphics in this paper, the ones that follow a small two-dimensional example (Figures 1, 3, 5, 6, 7 and 8), were added recently, as part of a review of the paper. Since the paper was written, lattice-based cryptography has moved from a research topic into standards: NIST has published lattice-based standards for key establishment and digital signatures (FIPS 203 and FIPS 204) [6]. The security of those schemes rests on the hardness of lattice problems, and algorithms of the kind studied here, the sieves for SVP and their descendants, are part of what is used to judge how hard those problems really are.

References

- [1] Miklós Ajtai, Ravi Kumar, and D. Sivakumar. A sieve algorithm for the shortest lattice vector problem. In *Proceedings of the 33rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 601–610, 2001.
- [2] Frank Arute et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.
- [3] László Babai. On Lovász’ lattice reduction and the nearest lattice point problem. *Combinatorica*, 6(1):1–13, 1986.
- [4] Arjen K. Lenstra, Hendrik W. Lenstra Jr., and László Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534, 1982.
- [5] Daniele Micciancio and Panagiotis Voulgaris. Faster exponential time algorithms for the shortest vector problem. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1468–1480, 2010.
- [6] National Institute of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard and FIPS 204: Module-lattice-based digital signature standard, 2024.
- [7] Phong Q. Nguyen and Thomas Vidick. Sieve algorithms for the shortest vector problem are practical. *Journal of Mathematical Cryptology*, 2(2):181–207, 2008.

- [8] Oded Regev. Lattices in computer science, 2004. Lecture notes, Tel Aviv University.
- [9] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.